

TP 2 - Architecture logicielle

Objectifs du TP :

- Comprendre l'utilité des design pattern
- Choisir le bon design pattern en fonction de la situation

Exercice 1

La municipalité d'une métropole vous a missionné pour concevoir le logiciel de gestion de sa flotte de drones de surveillance.

Le système doit pouvoir déployer des drones selon la zone d'intervention, sans que le centre de contrôle principal ne connaisse les détails techniques ou le modèle précis de chaque machine.

Il existe actuellement deux types de drones principaux : Le drone urbain, optimisé pour la ville, qui privilégie la discrétion et la légèreté. Le drone industriel, renforcé pour les zones de chantier, qui privilégie la résistance aux chocs et l'autonomie.

Le déploiement est géré par des centres de maintenance spécialisés. Le Centre de Maintenance Centre-Ville produit uniquement des drones urbains, tandis que le Centre de Maintenance Zone-Sud produit uniquement des drones industriels.

L'architecture doit permettre au centre de commandement global d'ordonner une patrouille en appelant une méthode `genererDrone()` sur un centre de maintenance, sans savoir si ce centre va lui fournir un modèle urbain ou industriel. L'objet retourné doit simplement répondre à une interface commune nommée `Drone`.

1. Identifiez le Produit Abstrait et les Produits Concrets dans ce scénario.
2. Identifiez le Créateur Abstrait et les Créateurs Concrets.
3. Réalisez le diagramme de classes UML correspondant en appliquant le design pattern Factory Method.
4. Expliquez comment ce pattern permet d'ajouter un nouveau type de drone (par exemple un drone aquatique) et un nouveau centre associé sans modifier le code du centre de commandement global.

Exercice 2

Une plateforme e-commerce vous demande de mettre en place un système d'alerte automatique pour la gestion de son inventaire. Dès qu'un article est en rupture de stock, plusieurs services doivent être informés simultanément pour réagir.

Actuellement, trois actions doivent se déclencher :

- Service Email : Envoie une notification immédiate au fournisseur.
- Tableau de bord : Affiche une alerte visuelle pour les logisticiens en entrepôt.
- Service Transport : Déclenche une demande de réapprovisionnement via une API externe.

L'architecture doit permettre à la classe Entrepot de signaler une rupture sans avoir à connaître les détails techniques de chaque service de notification.

Le système doit pouvoir accepter de nouveaux types d'alertes (SMS, Slack) sans modifier le code de gestion du stock.

1. Identifiez le Sujet (Observable) et les Observateurs (Observers).
2. Expliquez comment la classe Entrepot communique avec les services sans couplage fort.
3. Réalisez le diagramme de classes UML en appliquant le design pattern Observer.
4. Expliquez comment ce pattern respecte l'Open/Closed Principe (OCP) lors de l'ajout d'une alerte SMS.

Exercice 3

Pour chaque scénario, identifiez le design pattern concerné, et expliquez pourquoi.

Scénario A

Une entreprise de domotique crée une application pour piloter divers objets connectés : lampes, volets, et alarmes.

L'utilisateur doit pouvoir annuler sa dernière action (Undo) ou programmer une séquence d'actions à déclencher plus tard.

Le système doit transformer chaque requête (allumer, fermer, activer) en un objet autonome contenant toutes les informations nécessaires à son exécution.

Scénario B

Un logiciel de comptabilité internationale doit calculer le montant de la TVA pour une facture. Le calcul change radicalement selon le pays (France, USA, Japon).

Au lieu de remplir le code de conditions if/else géantes, le développeur souhaite que l'objet Facture puisse changer d'algorithme de calcul dynamiquement au moment de l'exécution, sans modifier son propre code source.

Scénario C

Un site de vente de matériel informatique permet aux clients de configurer leur propre PC.

Le processus de création d'un ordinateur est complexe : il faut installer le processeur, puis la RAM, puis la carte graphique, et enfin le système d'exploitation dans un ordre précis.

Le client veut simplement cliquer sur "Construire PC Gamer" ou "Construire PC Bureautique" sans gérer lui-même l'ordre des étapes de montage.

Exercice 4

Une coopérative apicole vous demande de développer le logiciel de contrôle d'une "Ruche Intelligente".

La ruche change radicalement de comportement selon son état biologique, et les actions de l'apiculteur (récolter, traiter, ouvrir) doivent s'adapter automatiquement à cet état.

Il existe trois états principaux :

- **Hivernage** : Les abeilles sont en grappe pour survivre au froid. La récolte est impossible et l'ouverture est interdite (risque de tuer la colonie par choc thermique).
- **Activité Standard** : Les abeilles butinent activement. La récolte est possible et l'inspection de routine est autorisée.
- **Essaimage** : La colonie est en crise, la reine s'apprête à partir avec la moitié des ouvrières. La récolte est bloquée, seule l'action "Capturer Essaim" est prioritaire.

L'objectif est de supprimer les structures conditionnelles massives dans la classe Ruche en déléguant le comportement à des objets "État" interchangeables.

1. Pourquoi le pattern State est-il plus adapté ici que de simples booléens ?
2. Créez l'interface EtatRuche et les classes Hiver, Saison, Crise.
3. Rendre fonctionnel et testez le menu interactif ci-dessous pour piloter la ruche.

```
Exemple en JAVA - Language de votre choix

public class RucheApp {
    public static void main(String[] args) {
        Ruche connectee = new Ruche(); // Initialement en Hivernage
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println("\n--- PANNEAU DE CONTRÔLE RUCHE ---");
            System.out.println("1. Tenter une récolte");
            System.out.println("2. Changer d'état (Passer en Été)");
            System.out.println("3. Quitter");

            int choix = scanner.nextInt();
            if (choix == 1) connectee.recolter();
            if (choix == 2) connectee.setEtat(new SaisonActive());
            if (choix == 3) break;
        }
    }
}
```

Exercice 5 (Bonus / Option)

Une agence de voyage vous demande de reprendre le code de son système de réservation de circuits touristiques (diagramme en fin de TP). Le code actuel est un "plat de spaghetti" où tout est mélangé dans une classe géante.

Vous devez identifier les problèmes de conception, renommer les éléments et proposer une architecture propre basée sur les Design Patterns.

Le diagramme actuel contient les éléments suivants :

Classe TrucManager : Cette classe possède une méthode `executer()` qui contient un énorme bloc `switch/case`. Selon que le client choisit un circuit "Éco", "Luxe" ou "Aventure", elle calcule le prix, réserve les hôtels et choisit le mode de transport (Bus, Jet Privé ou 4x4) directement dans le code.

Classe GrosObjetDonnees : Un objet massif qui contient à la fois le nom du client, la liste des 50 étapes du voyage, les coordonnées GPS, les factures PDF en binaire, les avis des clients et l'historique des modifications. Il est impossible de créer cet objet simplement.

Classe NotificationHandler : Cette classe est directement liée aux classes `EmailService` et `SMSService`. Si on veut ajouter une notification via une application mobile, il faut modifier le code interne de `NotificationHandler`.

1. Identifiez le design pattern de comportement qui permettrait de remplacer le `switch/case` de `TrucManager` pour la gestion des types de circuits. Identifiez le design pattern de création qui permettrait de construire l'objet `GrosObjetDonnees` étape par étape de manière sécurisée.
2. Identifiez le design pattern qui permettrait à `NotificationHandler` de prévenir plusieurs services sans être couplé à leurs classes concrètes.
3. Renommez ces trois classes avec des noms explicites respectant les standards du Clean Code. Proposez le diagramme de classes UML final corrigé

